DRIVER PULSE

System Design Document

Project	Driver Pulse — Rideshare Driver Shift Intelligence
Document Type	System Design Document (SDD)
Version	1.0
Prepared By	Team Popcorns
Date	10-03-2026
Status	Active — Prototype Implementation Complete

C O N T E N T S

1. Product Vision
2. User Persona
3. System Goals
4. Stress Detection Algorithm
5. Earnings Velocity Algorithm
6. Signal Fusion Logic
7. Real-Time Processing Strategy
8. Edge Processing Strategy
9. Scalability Plan
10. Execution Strategy
11. Cut Line

1 Product Vision

Mission Statement

Driver Pulse is a real-time shift intelligence system for rideshare drivers. It fuses three independent signal streams — motion sensor data, cabin audio intensity, and earnings telemetry — into a single, actionable dashboard that helps drivers stay safe, earn more, and understand their own performance across every trip in a shift.

The Problem

Rideshare drivers operate in a demanding environment where safety pressures and earnings pressures compound simultaneously. A driver braking hard in traffic, navigating a difficult passenger interaction, and watching their earnings fall behind their daily goal must process all of these signals without any system support. Existing driver apps show earnings but provide no insight into driving quality, stress accumulation, or pace-to-goal within a live shift.

The Solution

Driver Pulse instruments the driver's phone to passively capture motion and ambient audio intensity during trips. These signals are processed alongside trip earnings data to produce three categories of insight:

- Safety intelligence — which events during a trip were physically stressful, why they were flagged, and how they contribute to a cumulative shift stress score
- Earnings intelligence — whether the driver is on pace for their shift goal, what velocity they need to maintain, and which routes or time windows are most profitable
- Trip quality intelligence — a blended score combining safety and earnings signals that lets drivers identify their best and worst trips

Design Principles

Principle	Application in Driver Pulse
Explainability over black-box scoring	Every flagged event includes a plain-English description generated by build_explainability_log(). Drivers are never shown a number without context.
Privacy by architecture	Audio intensity (dB level) is captured, never raw audio waveforms. No voice content is stored or transmitted at any layer of the system.

Principle	Application in Driver Pulse
Graceful degradation	All engines use validate_schema() to warn on missing data rather than crash. Sentinel values preserve output shape when sensor data is absent.
Stateless, composable engines	Each engine is a pure DataFrame transformation. No shared state. Engines can be tested, replaced, or scaled independently.
Mobile-first UX	The Streamlit dashboard is constrained to a 480px max-width with Inter typography, touch-friendly tap targets, and dark-mode card layouts.

2 User Persona

Primary Persona: The Full-Time Driver

Rajan, 34 — Full-time rideshare driver, Bangalore
Vehicle: Maruti Suzuki Swift (4 years on platform)
Shift: 10 hours per day, 6 days per week
Daily earnings target: ₹1,800
Primary concern: Hitting daily goal while managing fatigue and difficult passengers
Pain point: No visibility into whether current pace will meet the day's target

Goals

- Know, at any point in the shift, whether earnings are on track
- Understand which trips and routes are most valuable
- Receive early warning when driving behaviour is accumulating stress
- Get specific, actionable guidance — not generic scores

Frustrations with Existing Tools

- The platform app shows total earnings but not velocity or pacing
- No feedback on driving quality until a formal complaint is filed
- No way to distinguish a profitable route from an average one in real time
- Stress from difficult passengers has no record or acknowledgement

Secondary Persona: The Part-Time Driver

Priya, 28 — Part-time driver, evenings and weekends
Vehicle: Honda City (1 year on platform)
Shift: 4–5 hours per day, 3 days per week
Daily earnings target: ₹800
Primary concern: Maximising earnings in a short window without overextending
Pain point: Does not know which hours of her shift are most productive

Goals

- Quickly assess whether the current shift is going well
- Identify the dead zone — when demand drops and it is better to stop
- Avoid accumulating stress-flagged events that could affect ratings

Non-Target Users

Driver Pulse is not designed for fleet managers or platform operations teams in this version. All insights are driver-facing and framed from the driver's perspective. A future fleet analytics layer is in scope for the post-hackathon roadmap.

3 System Goals

Functional Goals

Goal ID	Goal	Implemented
G-01	Detect harsh braking events from accelerometer data	Yes — sensor_engine.py
G-02	Detect sharp turn and swerve events	Yes — sensor_engine.py
G-03	Detect speed bump hits at non-zero speed	Yes — sensor_engine.py
G-04	Detect speeding above 60 km/h	Yes — sensor_engine.py
G-05	Detect jerk events (rate of change of acceleration)	Yes — sensor_engine.py
G-06	Detect cabin audio conflict signals	Yes — sensor_engine.py
G-07	Filter mechanical noise (door slams, road bumps)	Yes — sensor_engine.py
G-08	Compute per-trip stress score via signal fusion	Yes — sensor_engine.py
G-09	Calculate earnings velocity per trip and over shift	Yes — earnings_engine.py
G-10	Forecast goal completion against driver target	Yes — earnings_engine.py
G-11	Detect surge-inflated velocity and apply dampening	Yes — earnings_engine.py
G-12	Detect dead-zone conditions (low demand, low velocity)	Yes — earnings_engine.py
G-13	Generate motivational messages by shift phase	Yes — earnings_engine.py
G-14	Compute blended trip quality score	Yes — merge_engine.py
G-15	Generate driver shift summary with best route	Yes — merge_engine.py
G-16	Display all insights in mobile-first dashboard	Yes — app_realtime_demo.py (unified app)
G-17	Simulate real-time sensor streaming	Yes — app_realtime_demo.py (unified app, Live Sim tab)

Non-Functional Goals

Requirement	Target
Schema fault tolerance	Missing input columns emit [WARN] log; pipeline continues with sentinel values
Output completeness	Every driver in input produces at least one row in driver_shift_summary.csv
Sensor noise resilience	Z-score cap at 3.5σ; 15-second event cooldown; stationary suppression < 5 km/h
Privacy compliance	No raw audio captured; dB intensity only; on-device processing boundary enforced
Dashboard responsiveness	Streamlit cache TTL of None (indefinite per session, cleared on process restart); chart renders under 2 seconds on mid-range device
Code testability	All engines are pure functions accepting DataFrames; no global state; fully unit-testable

4 Stress Detection Algorithm

Overview

Stress detection is a two-phase pipeline. Phase 1 runs seven independent rule-based checks against raw accelerometer readings and four checks against audio intensity readings. Phase 2 fuses the binary check outputs into a continuous stress score using a weighted combination with a co-occurrence boost.

Phase 1A — Accelerometer Checks

All seven checks are applied independently to each sensor reading. A reading may trigger multiple checks simultaneously.

Check	Threshold & Logic
Harsh Brake	Longitudinal deceleration > 2.5 m/s^2
Sudden Acceleration	Longitudinal acceleration > 2.5 m/s^2
Sharp Turn / Swerve	Lateral vector magnitude $\sqrt{(x^2 + y^2)} > 2.0$ m/s^2
Speed Bump	Z-axis drop below 7.5 m/s^2 at speed > 15 km/h
Speeding	GPS speed > 60 km/h
Jerk	Rate of change of total magnitude > 3.0 m/s^3
Overall Magnitude	$\sqrt{(x^2 + y^2 + \lvert z - 9.8\rvert^2)} > 3.5$ m/s^2

Phase 1B — Audio Checks

Check	Threshold & Logic
Argument	dB level > 90 for any duration exceeding door slam threshold
Very Loud	dB level > 80 for any duration exceeding door slam threshold
Elevated / Sustained	Audio above baseline for > 60 continuous seconds
Door Slam Filter (exclusion)	Any spike with duration < 3 seconds is classified as mechanical noise and excluded

Pre-Check Filters

Before any check fires, three suppression filters are applied:

Filter	Logic
Stationary suppression	Readings at speed < 5 km/h are ignored (EC-06). Prevents false events during traffic queues, parking lot manoeuvres, and phone handling.
Trip boundary margin	Events within ACCEL_TRIP_MARGIN = 20 seconds of trip start or end are excluded. Handles pickup stop, door close, and dropoff braking.
Z-score outlier cap	ZSCORE_CAP = 3.5σ. Sensor values beyond 3.5 standard deviations are clipped to the cap value to suppress hardware malfunction spikes.

Phase 2 — Stress Score Fusion

Each flagged moment receives a combined stress score computed as:

```
combined_score = (motion_score × 0.65) + (audio_score × 0.35)
```

If both the motion score exceeds 0.4 AND the audio score exceeds 0.4 within the same sensor window, a 1.20× co-occurrence boost is applied:

```
if motion_fired and audio_fired:
    combined_score = combined_score × BOTH_BOOST  # 1.20
```

The motion weight (0.65) is higher because accelerometer events are more reliably indicative of objectively dangerous driving behaviour. Audio signals are weighted lower because they are more ambiguous — a loud radio and an argument produce similar dB readings at the intensity level.

Road Noise Suppression

If an audio spike co-occurs within a ROAD_NOISE_BOOST_WINDOW of 5 seconds of a confirmed speed bump event, the audio signal is suppressed from the stress score. This prevents road noise transmitted through the vehicle chassis from being misclassified as cabin conflict.

Cooldown Logic

A FLAG_COOLDOWN of 15 seconds is enforced between consecutive flagged moments on the same trip, regardless of event type. This prevents a single braking sequence or audio burst from generating dozens of flagged rows.

Severity Bands

Band	Score Range
NONE	0.00 – 0.29
LOW	0.30 – 0.49
MEDIUM	0.50 – 0.74
HIGH	0.75 – 1.00

> **Note — Two separate severity scales:** The table above shows the per-event severity bands used by _severity() in sensor_engine.py to label each individual flagged moment (none/low/medium/high). A separate STRESS_BADGE scale in merge_engine.py classifies the driver's overall peak shift stress score for the dashboard badge (NONE < 0.20, LOW 0.20–0.39, MODERATE 0.40–0.64, HIGH ≥ 0.65). These are distinct and should not be conflated.

Explainability Output

Every flagged moment is passed through build_explainability_log() which generates a plain-English description combining the motion event type, audio event type, and combined severity. Example output:

```
Serious event — Hard braking detected with loud argument in cabin.
Moderate event — Sharp turn or swerve with elevated noise levels.
```

5 Earnings Velocity Algorithm

Core Metric: Earnings Velocity

Earnings velocity is the primary financial metric in Driver Pulse. It expresses the rate at which a driver is accumulating earnings, normalised to an hourly rate:

```
earnings_velocity = fare / (duration_min / 60)  # ₹/hr
```

This metric is computed at two levels: per-trip (a snapshot of how efficient a single trip was) and as a rolling cumulative metric over the shift (used for goal forecasting).

Trip Earnings Rating

Rating	Velocity Threshold
excellent	>= ₹400 / hr
good	>= ₹280 / hr
average	>= ₹180 / hr
below_average	< ₹180 / hr

Shift Phase Model

The earnings engine divides a shift into four phases based on what fraction of the target hours have elapsed. Phase thresholds are used to contextualise forecast messages and tighten tolerance bands as the shift progresses.

Phase	Elapsed Fraction of Shift
early	< 25%
mid	25% – 59%
peak	60% – 79%
late	>= 80%

Forecast Status Logic

At each earnings velocity log entry, the engine computes the velocity needed to reach the target from the current position:

```
needed_velocity = (target_earnings - current_earnings) / remaining_hours
```

The ratio of current velocity to needed velocity is compared against a phase-specific tolerance band:

Status	Condition
achieved	current_earnings >= target_earnings
on_track	current_vel / needed_vel >= (1 - tolerance)
at_risk	current_vel / needed_vel >= (1 - tolerance - 0.20)
behind	current_vel / needed_vel < (1 - tolerance - 0.20)
dead_zone	effective_vel < area_median × 0.50 and remaining_hours > 1

Tolerance tightens as the shift progresses: 20% early, 15% mid, 10% peak, 5% late. A driver 18% below needed velocity at early phase is on_track. The same driver at late phase is behind.

Surge Dampening

When current velocity exceeds the driver's personal baseline median by more than 2×, the system detects a surge condition and applies a dampening blend to prevent overly optimistic forecasts:

```
if current_vel > baseline_vel × 2.0:
    effective_vel = (0.4 × current_vel) + (0.6 × baseline_vel)
```

This prevents the forecast from assuming surge pricing will continue for the remainder of the shift.

Dead Zone Detection

A driver is flagged as being in a dead zone when all of the following conditions are simultaneously true:

- Effective velocity < 50% of area median velocity
- Shift status is in_progress
- Remaining hours > 1

The dead zone signal surfaces in the dashboard as a specific motivational message encouraging the driver to relocate to a higher-demand area.

Motivational Message Engine

The _motivational_message() function generates shift-phase-aware messages that cite specific numbers wherever possible. Messages are never generic. Examples by status:

Status + Phase	Example Message
achieved	🎯 Goal reached! Any more trips today are a bonus.
on_track, mid	You're on track — just 40% left to go.
at_risk	⚠️ You need ₹85/hr more than your current pace. Try to fit in 3 more trips.
behind	📉 You're behind goal — currently ₹120/hr short of needed pace. Focus on 6 more trips in the next hour.

6 Signal Fusion Logic

Purpose

Signal fusion is the responsibility of merge_engine.py. It takes the independent outputs of the Sensor Engine and the Earnings Engine and joins them at the trip level to produce two composite artefacts: a per-trip quality score and a per-driver shift summary.

Trip-Level Join

The merge engine performs a left join of trip_earnings_df and sensor_summaries_df on trip_id. This preserves all trips in the earnings record, even if sensor data is absent for some trips.

```
merged = trip_earnings_df.merge(
    sensor_summaries_df[['trip_id', 'stress_score', 'flagged_count', ...]],
    on='trip_id', how='left'
)
```

> ⚠️ **Sentinel Design Decision**
>
> Trips with no sensor match receive stress_score = -1.0, not 0.0.
> This distinction is critical: 0.0 would be interpreted as a perfectly calm trip.
> -1.0 is a sentinel value treated as 'unknown' by _trip_quality(), returning 'average'.
> This prevents false-positive excellent trip ratings for trips with missing sensor data.

Trip Quality Matrix

The _trip_quality() function applies a two-dimensional decision matrix using stress score and earnings rating:

Stress Score	Earnings Rating	Trip Quality
== -1.0 (sentinel)	any	average
>= 0.65 (HIGH)	any	poor
< 0.20	excellent or good	excellent
< 0.35	excellent / good / average	good
< 0.50	any	average
>= 0.50	any	poor
any	below_average + stress < 0.35	average

Stress Score	Earnings Rating	Trip Quality
any	below_average + stress >= 0.35	poor

🔧 **Bug Fix: below_average Handling**

An earlier version of the merge engine had no explicit branch for below_average earnings rating.
This caused every low-earning trip to fall through to the 'poor' quality bucket,
incorrectly penalising drivers for slow periods even when driving was calm.
The fix adds an explicit branch: below_average + stress < 0.35 => average.

Driver Shift Summary Aggregation

build_driver_shift_summary() produces one row per driver containing:

Field	Source Logic
total_earned	Sum of fare across all trips
avg_velocity	Mean of earnings_velocity across all trips
peak_stress_score	Max stress_score across all trips
stress_badge	NONE / LOW / MODERATE / HIGH derived from peak_stress_score
n_excellent/good/average/poor_trips	value_counts() of trip_quality column
best_route	Route label from trip with highest earnings_velocity
forecast	From enriched_goals.py: on_track / at_risk / behind / achieved
goal_message	Phase-aware motivational message from earnings engine
total_flags / high_flags	Count of all flags and high-severity flags from flagged_moments.csv

🔧 **Bug Fix: None-Safe Goal Accessor**

An earlier version crashed with IndexError when a driver had no goal row in enriched_goals.
This occurred for new drivers or drivers with incomplete input data.
The fix introduces a gr = goal_row.iloc[0] if not goal_row.empty else None guard.
All downstream goal field accesses check gr is not None before reading.

7 Real-Time Processing Strategy

Current Implementation: Simulated Streaming

The app_realtime_demo.py application demonstrates the real-time processing architecture by replaying a pre-recorded trip dataset (realtime_trip_simulation.csv) via the Live Sim tab. New sensor readings are injected one row at a time at a 2-second interval. During replay, a lightweight inline detect_live_event() function runs threshold checks on each new row and updates charts and alert cards without invoking the full engine. The complete sensor_engine pipeline runs once at trip completion, producing a real trip summary identical to a batch-processed trip — demonstrating that the streaming and batch pipelines are architecturally identical.

This approach was chosen because it allows the full streaming architecture to be demonstrated without requiring a physical mobile device, live sensor SDK, or real trip data during the hackathon window.

Intended Production Architecture

In production, simulated replay would be replaced by a real-time sensor ingestion pipeline:

Layer	Production Component
Sensor capture	Mobile SDK polling accelerometer at 5 Hz, audio intensity at 1 Hz
Edge preprocessing	On-device: z-score capping, stationary filter, trip boundary detection
Stream transport	Compressed feature vectors pushed to Kafka / Google Pub-Sub topic per driver
Stream processor	Apache Flink or Spark Streaming consuming the topic, running engine logic
Output sink	Results written to time-series store (BigQuery / ClickHouse) and served via API
Dashboard	Streamlit or React native app polling the output API at 5-second intervals

Incremental Processing Design

The engines are designed to support incremental computation. Because each engine is a pure transformation on DataFrames with no shared state, new sensor readings can be appended to the input buffer and the affected trip's outputs can be recomputed without reprocessing the entire shift.

```
# Conceptual incremental update
new_reading = stream.next()
accel_buffer = accel_buffer.append(new_reading)
updated_trip_flags = run_accel_checks(accel_buffer.tail(WINDOW))
updated_sensor_summary = build_trip_summaries_sensor(trip, updated_trip_flags, ...)
```

Latency Budget

Stage	Target Latency
Sensor capture to edge buffer	< 50 ms
Edge preprocessing (outlier cap, filter)	< 100 ms
Stream transport (edge to cloud)	< 500 ms
Engine processing (sensor + earnings + merge)	< 200 ms per trip update
Dashboard refresh	Fragment-scoped rerun at 0.2s tick; row advances every 2s (prototype); production: WebSocket push
Total end-to-end (sensor to display)	< 6 seconds

8 Edge Processing Strategy

Motivation

Rideshare drivers operate in environments with intermittent connectivity, limited device battery, and strong privacy expectations around in-cabin audio. An edge-first processing strategy addresses all three constraints simultaneously: it reduces data transmitted over the network, preserves battery by avoiding continuous cloud round-trips, and ensures that raw sensor signals never leave the device.

On-Device Processing Boundary

Processed On-Device (Edge)	Sent to Cloud
Raw accelerometer readings (x, y, z, speed)	Aggregated feature vectors per event window
Raw audio dB samples	Audio event labels (argument / elevated / normal)
Z-score outlier capping	Capped and normalised values only
Stationary detection filter	Readings above MIN_SPEED_MOVING threshold only
Trip boundary margin suppression	Events outside the 20-second boundary only
Door slam classification (< 3s spikes)	Confirmed cabin audio events only

Battery Constraints

The accelerometer polling rate directly controls battery consumption. The system is designed around a configurable MIN_SPEED_MOVING threshold that suppresses all processing when the vehicle is stationary, which is the majority of urban driving time during traffic queues. This means the effective duty cycle is significantly lower than the nominal sampling rate.

Sampling Rate	Battery Impact	Detection Quality
50 Hz	High	Excellent — captures short-duration jerk events
10 Hz	Medium	Good — captures all braking and turning events
5 Hz (default)	Low	Good — sufficient for all implemented checks
1 Hz	Very low	Limited — may miss short jerk events

Offline Trip Processing

All engine logic runs on the local file system. Trips are processed offline and outputs are written to the local data directory. When connectivity is restored, outputs are synced to the cloud store. The pipeline makes no network calls and has no hard dependency on a live backend.

Memory Management

The sensor engine operates on rolling time windows rather than full-trip buffers. The stationary detection window (STATIONARY_WIN = 10 seconds) and cooldown window (FLAG_COOLDOWN = 15 seconds) define the maximum lookback required at any point. This bounds memory usage to a fixed window size rather than growing proportionally with trip length.

9 Scalability Plan

Current Scale

The prototype processes a single driver's shift data synchronously in a single Python process. The Streamlit cache (TTL = None, indefinite per session) prevents redundant reprocessing on tab switches or chart interactions. All data is stored in CSV files on the local filesystem.

Scale Targets (Post-Hackathon)

Dimension	Target
Concurrent drivers	10,000+ simultaneous active trips
Sensor event throughput	50,000+ events per second across fleet
Dashboard latency	Sub-10-second end-to-end from sensor to display
Data retention	90 days of per-driver shift history
Output availability	99.9% uptime for the driver dashboard API

Horizontal Scalability

The three-engine pipeline is stateless and embarrassingly parallel at the driver level. Each driver's shift can be processed independently with no inter-driver dependencies. This means the processing layer can be scaled horizontally by partitioning the driver population across multiple worker instances.

```
# Kafka partition key = driver_id
# Each partition processed by independent worker
# No cross-partition state required
```

Infrastructure Roadmap

Phase	Component	Technology
Phase 1 (Current)	Single-process pipeline	Python + Pandas + Streamlit
Phase 2	Stream ingestion layer	Apache Kafka / Google Pub-Sub
Phase 3	Distributed stream processing	Apache Flink or Spark Streaming

Phase	Component	Technology
Phase 4	Time-series output store	BigQuery / ClickHouse
Phase 5	ML inference layer	TensorFlow Serving / Vertex AI
Phase 6	Mobile-native dashboard	React Native embedded in driver app

Database Strategy

Data Type	Storage Strategy
Raw sensor events (write-heavy)	Append-only Kafka topic retained for 7 days
Flagged moments (queryable)	ClickHouse columnar store partitioned by driver_id and date
Shift summaries (low volume)	PostgreSQL with driver_id + shift_date composite index
Earnings velocity log	Time-series store (InfluxDB or BigQuery) with hourly rollups
Driver profiles and goals	PostgreSQL with driver_id primary key

10 Execution Strategy

Hackathon Execution

The hackathon deliverable was scoped to demonstrate the full end-to-end architecture at prototype fidelity real signal processing logic, real earnings forecasting, real signal fusion using simulated data in place of live sensors and APIs. The following decisions were made deliberately to fit the hackathon window:

Decision	Rationale
CSV-based data pipeline	Eliminates infrastructure dependencies; focuses evaluation on signal processing logic
Rule-based detection over ML	Interpretable, debuggable, requires no training data; ML is scoped to Phase 2
Streamlit dashboard	Full Python stack; rapid iteration; mobile-first CSS achieves native-app appearance
Simulated real-time replay	Demonstrates streaming architecture without requiring physical device or live API
Synthetic trip dataset	Ensures reproducible demonstration; avoids PII handling requirements

Post-Hackathon Execution Plan

The following roadmap defines the path from hackathon prototype to production deployment:

Milestone	Deliverable
M1 — Mobile SDK Integration	Replace CSV ingestion with live accelerometer and audio intensity SDK. Android first, iOS to follow.
M2 — Stream Infrastructure	Deploy Kafka ingestion layer and Flink processor. Validate < 6-second end-to-end latency at 100 concurrent drivers.
M3 — Platform API Integration	Connect to ride-hailing platform webhooks for real trip events, fare data, and surge multipliers.
M4 — Driver Profile Service	Per-driver baseline calibration. Thresholds adapt to individual driving style, vehicle type, and city.
M5 — ML Stress Classifier	Train supervised classifier on labelled event data. Replace rule-based checks for harsh braking and conflict detection.

11 Cut Line

What Is Built

✅ Implemented and Demonstrated

7 accelerometer checks + 4 audio checks in sensor_engine.py
Stress score fusion with co-occurrence boost and road noise suppression
Z-score outlier capping, stationary suppression, trip boundary margin, cooldown logic
Explainability log with plain-English event descriptions
Earnings velocity calculation per trip and per shift
Shift phase model (early / mid / peak / late) with tightening tolerance bands
Forecast status (on_track / at_risk / behind / achieved / dead_zone)
Surge dampening via 2x baseline threshold and 40/60 velocity blend
Dead zone detection based on area median velocity
Phase-aware motivational message generation
Signal fusion via trip-level left join with sentinel stress score
Trip quality matrix with below_average earnings bug fix
Driver shift summary with best route, stress badge, and flag counts
Unified mobile-first Streamlit dashboard with Live Sim tab, full analytics, and real-time simulation demo (app_realtime_demo.py — single entry point; app.py has been removed)

What Is Explicitly Deferred

⬜ Not Built — Post-Hackathon Roadmap

Live sensor SDK (accelerometer and audio from physical device)
Real ride-hailing platform API integration (trip webhooks, fare data, surge)
Kafka / Flink stream processing infrastructure
Per-driver ML-based stress classifier
Deep audio classification model (argument vs. music vs. conversation)
React Native or native mobile app deployment
Per-driver baseline calibration and personalised thresholds
Fleet analytics layer (aggregate insights for fleet managers)
Push notification service for real-time alerts
Data retention infrastructure (BigQuery / ClickHouse)
Driver consent framework and privacy audit for audio intensity collection

Known Limitations of the Prototype

Limitation	Impact & Mitigation
Synthetic data only	Pipeline correctness validated; real-world threshold calibration requires labelled field data
Global detection thresholds	Thresholds are not calibrated per driver or vehicle type; per-driver baselines are a Phase 2 item
No real-time sensor stream	Simulated replay demonstrates architecture; swap-out requires only replacing ingestion layer
Streamlit, not native mobile	Full functionality delivered; mobile-first CSS mitigates UX gap; React Native migration is planned
Single-process pipeline	Sufficient for prototype; horizontal scaling via Kafka partitioning is architecturally ready
CSV output, not database	All output shapes are production-compatible; database migration requires no engine changes

📋 Evaluation Note for Judges

Every component above the cut line is functional, tested, and demonstrated in the prototype.
Components below the cut line are documented in the architecture (README.md, design diagrams) to show production readiness of thinking, not to claim they are implemented.
The distinction between built and deferred is explicit and intentional.